

HU-SPIKE – Análisis de enfoques RAG y decisión técnica para el modelo teacher con RAG

Documento de síntesis, comparación y recomendación técnica

Autor	Alejandro Tirado Ramirez	Fecha	22/03/2026
Sprint	1	Estado	Done

1. Descripción de la HU

Como investigador, quiero analizar enfoques de RAG y sus variantes, para diseñar un modelo teacher con RAG alineado al estado del arte.

2. Objetivo de la HU

Identificar, sintetizar y comparar las principales arquitecturas y variantes de RAG relevantes para el proyecto, con énfasis en su aplicabilidad a un corpus legal en español, de modo que el equipo pueda seleccionar una arquitectura teacher técnicamente justificada, viable de implementar y coherente con la línea experimental de destilación.

3. Contexto

El proyecto busca comparar una destilación baseline contra una destilación guiada por un teacher con RAG. En este contexto, la arquitectura RAG no puede definirse de forma intuitiva ni meramente teórica: debe sustentarse en literatura reciente, en las necesidades del dominio legal y en la evidencia técnica ya disponible en el prototipo.

Esta HU se enfoca en tomar una decisión documentada: comparar alternativas de RAG, entender sus *trade-offs* y recomendar el enfoque más apropiado para el teacher del proyecto.

4. Criterios de aceptación

Código	Criterio
CA1	Debe existir un documento de síntesis que describa los principales enfoques y variantes de RAG y sus características.
CA2	Debe existir una matriz de decisión que compare las arquitecturas consideradas según criterios definidos.
CA3	Debe quedar documentada la decisión del enfoque a implementar y su justificación técnica.
CA4	La selección del enfoque RAG debe respaldarse con fuentes académicas y técnicas del estado del arte.

5. Análisis comparativo

5.1. A. RAG denso simple (vector DB + top-k semántico)

El RAG denso simple corresponde a la arquitectura más básica de recuperación aumentada por generación. En este enfoque, los documentos se fragmentan en *chunks*, cada fragmento se transforma en un *embedding* y se almacena en una base vectorial. Cuando llega una consulta, esta también se convierte en un vector y se recuperan los top-k fragmentos semánticamente más cercanos según una métrica de similitud, como coseno o producto punto. Su principal ventaja es la simplicidad de implementación, ya que requiere pocos componentes y se integra fácilmente con *pipelines* modernos de *embeddings* y bases vectoriales. Sin embargo, su desempeño depende fuertemente de la calidad de los *embeddings* y del proceso de *chunking*. Además, puede recuperar fragmentos que son semánticamente parecidos, pero no necesariamente los más útiles o precisos para responder la consulta, lo que limita la calidad final del contexto entregado al modelo generativo.

5.2. B. RAG de dos etapas: recuperación densa + reranker

El RAG de dos etapas introduce una mejora sobre el esquema básico al separar la recuperación en dos momentos. En la primera etapa, se realiza una búsqueda densa para obtener un conjunto inicial de candidatos relevantes de forma eficiente. En la segunda, un modelo *reranker* vuelve a evaluar esos candidatos comparando cada fragmento con la consulta de manera más fina, y reordena los resultados según su relevancia real. Esta arquitectura busca equilibrar eficiencia y calidad: la recuperación densa permite reducir rápidamente el espacio de búsqueda, mientras que el *reranker* mejora la precisión del contexto final. En la práctica, este enfoque suele producir respuestas mejor fundamentadas, porque disminuye la probabilidad de que el modelo reciba fragmentos parcialmente relacionados, pero poco informativos. Su costo es mayor que el de un RAG simple, ya que añade una etapa extra de cómputo, pero sigue siendo una alternativa muy viable cuando se busca mejorar la calidad del *retrieval* sin rediseñar completamente la arquitectura.

5.3. C. RAG híbrido: recuperación densa + sparse/BM25 + reranker

El RAG híbrido combina distintos mecanismos de recuperación para aprovechar sus fortalezas complementarias. Normalmente integra una búsqueda densa, que captura similitud semántica, con una búsqueda *sparse* o léxica, como BM25, que resulta especialmente útil cuando la consulta contiene términos clave exactos, nombres propios, referencias normativas o expresiones técnicas poco frecuentes. Los resultados de ambas búsquedas se fusionan para construir un conjunto candidato más robusto, y luego un *reranker* se encarga de priorizar los fragmentos más relevantes. Esta estrategia suele mejorar el *recall*, porque no depende únicamente de la representación semántica de los *embeddings*, sino que también preserva coincidencias léxicas importantes. Esto es particularmente valioso en dominios especializados, como el legal o el normativo, donde una palabra exacta, un artículo o una entidad específica pueden ser determinantes. Su desventaja es que incrementa la complejidad de implementación y ajuste, ya que requiere coordinar múltiples recuperadores, definir una estrategia de fusión y manejar una latencia mayor.

5.4. D. RAG adaptativo/correctivo (Self-RAG / CRAG)

Las variantes adaptativas o correctivas representan una evolución más avanzada del paradigma RAG. En lugar de aplicar siempre el mismo flujo de recuperación, estos enfoques incorporan mecanismos para decidir cuándo recuperar, cómo evaluar la calidad de la evidencia obtenida y, en algunos casos, cómo corregir o refinar el proceso antes de generar la respuesta final. En arquitecturas como Self-RAG, el sistema puede aprender a autorregular el uso de recuperación y a reflexionar sobre la suficiencia del contexto. En propuestas como CRAG, se introduce una fase de verificación o corrección de la evidencia recuperada, con el fin de reducir errores y alucinaciones. Conceptualmente, estas variantes buscan que el sistema no solo recupere información, sino que también tenga una capa adicional de control sobre la confiabilidad del contexto. Aunque prometen mejor robustez y mayor alineación con tareas complejas, implican una complejidad técnica considerablemente mayor, tanto en diseño como en entrenamiento y evaluación. Por eso, suelen ser más adecuadas como líneas de evolución futura que como primera implementación en proyectos donde todavía se está consolidando una base RAG funcional.

5.5. Matriz de comparación

Alternativas consideradas

Código	Alternativa
A	RAG denso simple (vector DB + top-k semántico)
B	RAG de dos etapas: recuperación densa + reranker

Código	Alternativa
C	RAG híbrido: recuperación densa + sparse/BM25 + reranker
D	RAG adaptativo/correctivo (Self-RAG / CRAG)

Comparación por criterios

Criterio	A	B	C	D	Observaciones
Complejidad de implementación	Baja	Media	Alta	Alta	B es viable con el prototipo actual; C y D exigen más componentes y orquestación.
Calidad esperada del ranking	Media	Alta	Alta	Alta	B mejora el orden de los pasajes; C puede ganar <i>recall</i> ; D añade control adaptativo.
Latencia de inferencia	Baja	Media	Media-Alta	Alta	El <i>reranking</i> y la lógica adaptativa agregan costo.
Compatibilidad con el código actual	Alta	Muy alta	Media	Baja	El código ya usa Chroma, <i>embeddings</i> y <i>reranker</i> ; B requiere menos <i>refactor</i> .
Alineación con estado del arte	Media	Alta	Muy alta	Alta	<i>Reranking</i> y fusión de recuperación son prácticas ampliamente respaldadas; D es moderno pero más costoso.
Riesgo técnico	Bajo	Medio	Medio-Alto	Alto	D depende de componentes adicionales y mayor complejidad experimental.
Capacidad de evolución futura	Media	Alta	Muy alta	Alta	B deja una ruta limpia para evolucionar a C si la recuperación inicial es insuficiente.

5.6. Síntesis de enfoques analizados

- **RAG denso simple.** Recupera pasajes mediante *embeddings* y búsqueda vectorial. Es la opción de menor costo de implementación, pero depende con fuerza de la calidad del primer ranking y suele degradarse cuando el top-k contiene evidencia parcialmente relevante.
- **RAG de dos etapas con reranker.** Mantiene una primera fase de recuperación eficiente y agrega un modelo de *reranking* que reordena los pasajes candidatos según relevancia consulta-documento. Esta arquitectura mejora precisión contextual sin rehacer por completo el *pipeline*.

- **RAG híbrido con reranker.** Combina señales densas y léxicas/*sparse* para mejorar *recall*, especialmente cuando las consultas contienen nombres exactos, números de ley o referencias normativas. Luego aplica *reranking* sobre el conjunto fusionado.
- **RAG adaptativo/correctivo.** Incluye variantes como Self-RAG y CRAG, en las que el sistema decide cuándo recuperar, cómo criticar lo recuperado o cuándo corregir el contexto. Representa una frontera avanzada del estado del arte, pero eleva notablemente la complejidad de entrenamiento e inferencia.

5.7. Referencias consideradas

- R1 Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
- R2 Wu et al. (2024). *Retrieval-Augmented Generation for Natural Language Processing: A Survey*.
- R3 Asai et al. (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*.
- R4 Yan et al. (2024). *Corrective Retrieval Augmented Generation*.
- R5 Chen et al. (2024). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*.
- R6 BAAI. *bge-reranker-v2-m3 model card*.
- R7 Edge et al. (2024). *A Graph RAG Approach to Query-Focused Summarization*.
- R8 DeepEval documentation: contextual relevancy, contextual recall y contextual precision.

6. Resultado / Conclusión

6.1. Hallazgos principales

- El RAG denso simple es útil como *baseline*, pero deja demasiado peso en la calidad del primer ranking.
- Las arquitecturas de dos etapas con *reranker* representan un punto medio sólido entre calidad y factibilidad.
- Las variantes híbridas y adaptativas ofrecen ventajas potenciales, pero incrementan complejidad, costo de experimentación y esfuerzo de integración.
- La literatura reciente favorece *pipelines* con mejor control sobre recuperación y ranking, más que una simple recuperación top-k sin refinamiento.

6.2. Decisión técnica

Se va a adoptar como enfoque un RAG de dos etapas: recuperación semántica inicial sobre Chroma usando *embeddings*, seguida de *reranking* antes de construir el contexto final del *prompt*. Esta arquitectura debe considerarse la opción base del teacher con RAG.

La decisión se justifica porque: (1) es consistente con el prototipo actual; (2) incorpora una práctica fuertemente respaldada por el estado del arte reciente; (3) mantiene una complejidad manejable para el proyecto; y (4) deja preparada una ruta de mejora hacia recuperación híbrida si los experimentos muestran límites de *recall* en el corpus legal.

En otras palabras, la arquitectura seleccionada no es el RAG más simple posible, pero tampoco la variante más costosa del estado del arte. Es la mejor relación entre evidencia, madurez técnica, compatibilidad con el código y valor experimental para la destilación.

6.3. Impacto en el proyecto

- Permite construir un teacher con *grounding* documental más robusto que un *baseline* puramente paramétrico.
- Facilita la comparación experimental entre destilación *baseline* y destilación con RAG.
- Mantiene trazabilidad de fuentes y una arquitectura razonable para evaluación con métricas de contexto.

6.4. Trade-offs

Qué se gana	Qué se sacrifica	Riesgos
Mejor ordenamiento del contexto, mayor relevancia del top final y mejor alineación con prácticas modernas de RAG.	Más latencia y más complejidad que un <i>retrieval</i> simple.	Si el <i>reranker</i> no entra realmente al contexto final o si el <i>recall</i> inicial es bajo, la ganancia esperada se reduce.

6.5. Nivel de confianza

Nivel de confianza: Medio-Alto. La decisión está bien respaldada por literatura y por el estado actual del prototipo; sin embargo, la conveniencia de extender posteriormente a recuperación híbrida debe validarse empíricamente sobre el corpus legal del proyecto.

6.6. Recomendación final

- Consolidar primero el *pipeline* de dos etapas haciendo que el contexto del *prompt* use explícitamente los *chunks* rerankeados.
- Mantener BGE-M3 como base por su capacidad multilingüe y por su potencial de evolución hacia recuperación híbrida.
- Planear un experimento posterior para comparar *dense + rerank* versus *híbrido + rerank* si se detectan fallos de *recall*.

7. Próximos pasos

- Empezar con la implementación del RAG.
- Consultar y generar medidas para medir el desempeño del RAG.
- Realizar casos de prueba para asegurarnos del correcto funcionamiento del RAG.